

Neurocore: A GNN Approach to Configurable IP Core Identification in FPGA Netlists

Dallin Dahl, Keenan Faulkner, James Usevitch, and Jeffrey Goeders

Department of Electrical and Computer Engineering

Brigham Young University

Provo, Utah, USA

djdahl@byu.edu, keenanrf@student.byu.edu, james_usevitch@byu.edu, jgoeders@byu.edu

Abstract—Netlist reverse engineering enables many applications, including detecting IP theft, verifying CAD tool correctness, and detecting hardware trojans. However, reconstructing high-level information and circuit structure from a flat, nameless netlist is challenging. In this work we focus on the problem of locating known IP cores in an FPGA netlist, which is especially challenging due to the prevalence of highly configurable IP cores. We present Neurocore: a graph neural network-based approach to classifying nodes in a netlist as instances of known IP cores, and present and evaluate different models for different use cases. We have created a large open-source dataset of thousands of FPGA designs with known IP cores, and demonstrate that our approach can obtain 80–98% accuracy, depending on the model and the use case.

I. INTRODUCTION

Netlist reverse engineering is the process of documenting the structure and intent of a hardware design starting with a low-level netlist. This netlist could be recovered from design files, physical examination of a circuit (i.e. by delayering), or extracted from an FPGA bitstream. Although reverse engineering enables malicious behavior like IP theft and Trojan insertion, it is a critical component in many important and ethical practices as well. These practices include hardware Trojan detection, design equivalence checking, IP theft detection, and design recovery in the face of missing or obsolete source files, among others [1].

Even though a low-level physical netlist may simulate identically to the original design, it is missing critical information about the high-level structure and intent of the design. As an example, there are no names in a physical netlist, while those names would be present and likely very informative in the original source. Further, the netlist is missing information regarding hierarchy, bus groupings and control/data distinctions.

Previous work has focused on deriving this information. In [2], the authors focused on recovering known circuit modules. The authors in [3] focused on locating common large macros including register files, counters, adders and subtractors. Other work has looked at grouping signals into words ([4], [5], [6]). In [7], the authors look for even larger known circuit structures, and focus on identifying FPGA intellectual property (IP) cores using analytical methods.

These methods typically involve matching against precise features, either directly, or through some form of property hashing. This is fast and effective at finding exact matches

of, or small variations on a given circuit. However, such approaches are typically less effective at dealing with larger circuits, such as entire IP blocks, various configurations of IP, or IP that have been more substantially optimized by the CAD tools.

Successfully detecting known IP cores can help in reverse engineering as it can identify large pieces of an overall design, and reduce the amount of unknown structure that needs to be analyzed. This can reduce cost of manual labor that is often needed to fully understand a design.

In FPGA design, use of IP cores is common, and FPGA vendors include a large library of IP with their provided CAD tools. While finding a specific subcircuit in a netlist is a straightforward graph matching problem, in this work, we opt to tackle a more difficult problem: detecting configurable IP cores in a netlist.

The configurable nature of FPGA IP cores means that detecting them, and locating their cells in the netlist, requires a more sophisticated approach that can account for IP configurations that may not have been seen before. For example, Figure 1 shows a subset of the configuration options for the accumulator component, available in the AMD Vivado tool (one of two configuration tabs are shown in the figure). This IP has about 3.6×10^{10} possible configurations, which makes it impossible to enumerate all possible configurations and match them exactly. Some IP we investigate in this work have even more configurations, for example, the MicroBlaze has about 2.8×10^{34} configurations, while others have only a handful of configurations.

These configuration options allow for control over which features are enabled in the IP, performance targets, resource usage types, widths of data paths, presence of bus and other I/O connections, and more. For IP with a few configuration options, it may be possible to enumerate all possible combinations, and look for exact matches of the netlist structure. However, for many IP cores, the number of different configuration options multiplies the IP configuration space to be too large to enumerate.

Along with these configuration options, the cross-boundary optimizations that can take place during compilation further mutate the properties of the IP cores, and may make exact matching approaches less effective, even on IP configurations that have been seen before. This means that a solution to IP

detection must be capable of detecting similar circuit structure, rather than exact matches.

In this work we present and evaluate *Neurocore*, a tool for detecting *configurable* IP cores, which are common in FPGA designs. We use graph neural networks (GNNs) to perform a fuzzy matching of IP cores, which can tolerate variations in the IP and the surrounding circuit.

Neural networks are of course very popular in recent years, and have been shown to be quite effective at recognizing patterns in data, including recognizing and classifying data that does not have an exact match in the training data. However, such an approach requires substantial training data, and so a major focus of this work has been on creating a tool for generating randomized IP circuits.

The main contributions of this work are as follows:

- An approach to configurable IP circuit recognition using a graph neural network.
- Two open datasets of 5,000 circuits each.
- Experimental study on the performance of three networks in two different use cases, comparing them to results in prior work.

II. BACKGROUND

A. Graph Neural Networks

Graph Neural Networks (GNNs) are a class of deep learning models designed to work directly with graph-structured data. Unlike traditional neural networks, which are typically designed for fixed-size, grid-like structured data, GNNs are built to handle the more complex structure and dynamic size of graphs, which makes them suitable for circuit analysis.

GNNs are commonly used to perform either graph-level classification or node-level classification. In graph-level classification, the goal is to classify the entire graph as a whole, based on the features of the nodes and edges in the graph. For a circuit represented as a graph, this might be used to classify the entire circuit as a certain type of circuit, based

on the features of the nodes and edges in the graph. In node-level classification, the goal is to classify each node in the graph individually, based on the features of the node and its neighbors. For a circuit represented as a graph, this might be used to classify each cell in the circuit as a certain type of cell, based on the features of the cell and its neighbors.

In this work, we are interested in distinguishing between different types of IP in a circuit, so we elect to use a node-level classification approach, with the goal of classifying each cell in the circuit as a certain type of IP. It is worth mentioning that some cells in the circuit may not be part of any IP, and we want to be able to classify these cells as not belonging to any IP; this aspect has actually proven to be one of the most challenging aspects in finding a good GNN model for this task.

B. Related Work

There have been several previous works that have focused on gaining higher-level understanding of circuits from a flat, nameless netlist, such as one obtained from an FPGA bitstream, or from reverse engineering an ASIC.

In [3], the authors focus on recovering specific common circuit structures, including register files, counters, adders, and subtractors. They do this analytically, with a combination of structural and behavioral analyses. The projects in [4], [5], and [6] focus on word reconstruction algorithms. In the case of WordRev [4], this is done using a behavioral analysis, while the other two rely on structural features of the netlist. While the focus in this work is on labeling all nodes in a graph, the focus of Li et al. is identifying the function of a given subgraph from simulation. The work in [2] presupposes a division of a circuit into independent modules, and requires simulating each module. In contrast, this work takes a complete circuit and annotates each node based exclusively on structural information present in the netlist.

The closest work to this paper was done by Dahl et al. in 2023 [7]. They present two algorithms, one focused on partitioning a circuit into labeled subgraphs (*IPRec*), and one focused on labeling nodes (*Isoblaze*). *IPRec* focuses on directly matching various submodules of an IP in the circuit graph, whereas *Isoblaze* uses local neighborhoods and hashing to greatly accelerate the matching of circuits, at the cost of direct configuration matching. Both proceed in an analytical fashion. The *IPRec* algorithm scales poorly with the size of the graph to be analyzed due to its reliance on a backtracking exploration of the matching space. In [7], *IPRec* was unable to match the AXI interconnect and Microblaze IP, even with hours and days of runtime. The *Isoblaze* algorithm was reportedly much faster, but the hashing-based matching approach was unable to cope well with changes introduced by configuration options or optimization, resulting in average F1-scores of 0.32.

In addition, while the authors in [7] trained the analytical algorithms on many IP variations, the evaluation was quite limited, with only a single IP matched per inference run, and matching performed on only one evaluation circuit containing

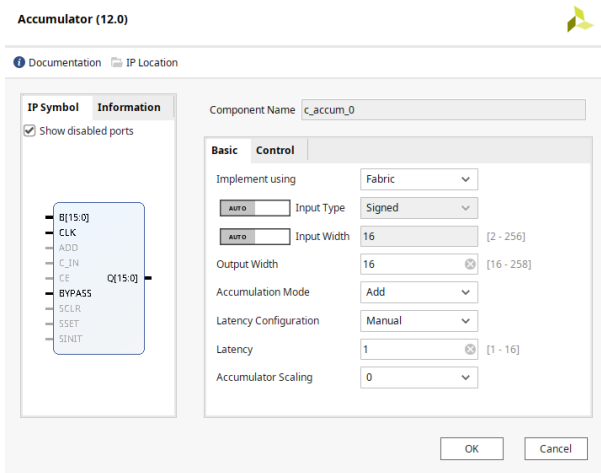


Fig. 1: Accumulator configuration options in Vivado

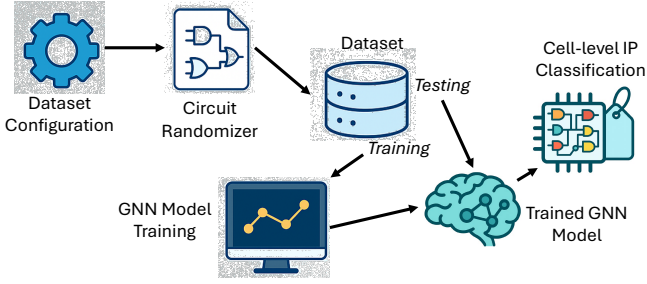


Fig. 2: IP detection and location using a GNN model

five IP. In contrast, this work focuses on creating large training and test datasets, which consist of fully implemented circuits containing up to 18 instances of IP from 16 different IP types. We evaluate our IP classification performance on validation datasets consisting of 1,000 designs each.

III. APPROACH

This work is targeted at answering two major research questions:

Research Question 1: How well do GNNs perform at identifying configurable IP in a netlist comprised largely of known IP?

Research Question 2: How do GNNs perform when the design contains significant amounts of IP or other logic, that is not present in the training data?

Figure 2 gives an overview of the system we’ve developed to address these questions. The major components of the system are as follows:

Dataset Generation First, a dataset containing a large number of circuit netlists is generated using a circuit generation tool we created. These designs contain a variety of IP cores, as specified with a configuration file, and each IP instance is configured randomly. The dataset is split into training and validations sets. This circuit generation process and resulting datasets are discussed in Section IV. The configuration of these datasets corresponds to the characteristics of each research question, as elaborated in Section V.

GNN Model Training and Evaluation Next, a GNN model is trained using the set of circuits in the training portion of the dataset. This training data contains a golden labeling of each Basic Element of Logic (BEL) (eg. LUT, flip-flop, etc.), indicating which IP core it belongs to. The trained model is then evaluated against the validation set. In the inference phase, the trained GNN model is fed a circuit netlist in graph form, and outputs an IP label for every node in the graph (nodes correspond to BELs in the netlist). This is discussed further in Section VI.

This architecture results in a single model trained to recognize the location of several IP simultaneously, while previous work [7] required much longer run times, and could only infer the location of one IP core at a time.

IV. DATASETS

One of the primary difficulties in using neural networks is the large datasets required to train the networks effectively. In other applications, datasets can be collected from the vast amounts of publicly available data online. However, in training our networks to identify specific classes of IP, we needed to have various configurations of the IP in various contexts, as well as a way to automate the labeling of the IP within the circuit. This necessitated creating our own datasets. As we developed the Neurocore tool, we developed a tool to automatically generate designs with Vivado’s IP Integrator tool. This allows us to create two large datasets that focused on the following characteristics:

- 1) IP in the dataset have nearly-full coverage of the configuration space for each IP.
- 2) IP are instantiated in context, with surrounding circuitry, allowing for boundary optimizations.
- 3) Dataset designs can contain multiple instances of a given IP, each with their own configuration.

Our design generator is fully automated, with a large degree of randomness. Although the designs it creates are not real designs, and would likely not be useful for any practical purpose, we have made some attempt to construct them in a way that has more realism than a completely randomly stitched together set of IP cores, as has been done in some previous work [8], [9], as well as more relevant than a realistic but arbitrary circuit [10]. For example, we make efforts to track whether signals are clocks, resets, interrupts, buses, control or data signals, and connect them to like signal types. We also support AXI bus connections, and can attach IP cores to the bus, and connect them in a valid manner.

Each design is created by 1) randomly selecting a number of IP cores from a set of IP cores, 2) randomly configuring each IP core instance, and 3) connecting the IP together.

Our two flagship datasets consist of 5,000 designs each, detailed in Table I. In *Dataset 1*, the training and validation designs choose from the same set of IP cores, which consist of 16 different AMD-provided IP cores. In *Dataset 2*, the validation designs contain IP cores that are not present in the training designs. This was done to test the ability of the model to handle designs with IP or other logic that is not seen during training. In this dataset, seven IP cores are withheld from the training set.

Table II shows the wide range of configurations of the various IP, with statistics from *Dataset 1*. The first data column indicates the number of total instances of the IP core across the 4,000 training designs, and the second column indicates the number of unique configurations. The next columns list the same statistics for the 1,000 test designs, and the final column indicates the number of configurations in the test set that are not present in the training set. For designs with large configuration spaces (e.g. MicroBlaze) that reach 100%, each IP configuration in the test set has never been seen in the training set. There is of course substantial common circuitry between different configurations, making this machine learning-based

TABLE I: Neurocore Datasets

Dataset Name	Train/Test	# Designs	GPIO	Uartlite	Accumulator	AXI Timer	DFT	MicroBlaze	AXI CDMA	AXI ICAP	EMC	XADC	AXI EthernetLite	AXI IIC	AXI QuadSPI	*ClkGen	*Interrupt Controller	* AXI SmartConnect
Dataset 1	Train	4,000	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
	Test	1,000	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
Dataset 2	Train	4,000	X	X	X	X	X	X								X	X	X
	Test	1,000	X	X	X	X	X	X								X	X	X

Each design composed of 3–18 randomly selected IP. ICAP and XADC have max of 1 instance.

*These IP are not randomly configured, but added to the design as needed to support the other IP cores.

TABLE II: Dataset 1 IP Configuration Uniqueness

IP	Training Set		Test Set		Unique not in Training
	Total Instances	Unique Instances	Total Instances	Unique Instances	
QuadSPI	2,657	956	681	422	159 (38%)
GPIO	3,240	2,336	832	700	494 (71%)
Accumulator	3,245	3,236	836	833	833 (100%)
DFT	3,274	264	858	248	0 (0%)
Uartlite	3,256	144	801	144	0 (0%)
MicroBlaze	2,996	2,996	774	774	774 (100%)
EMC	3,282	8	805	8	0 (0%)
XADC	2,100	2,083	554	552	539 (98%)
Timer	3,272	64	829	61	0 (0%)
ICAP	1,982	551	488	258	31 (12%)
CDMA	3,336	1,744	845	654	255 (39%)
EthernetLite	3,345	96	826	96	0 (0%)
IIC	3,411	3,411	871	871	871 (100%)

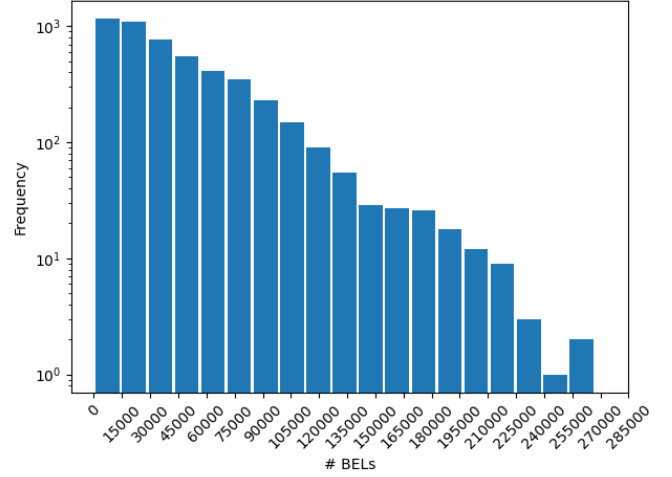


Fig. 4: Dataset 1: Distribution of design sizes

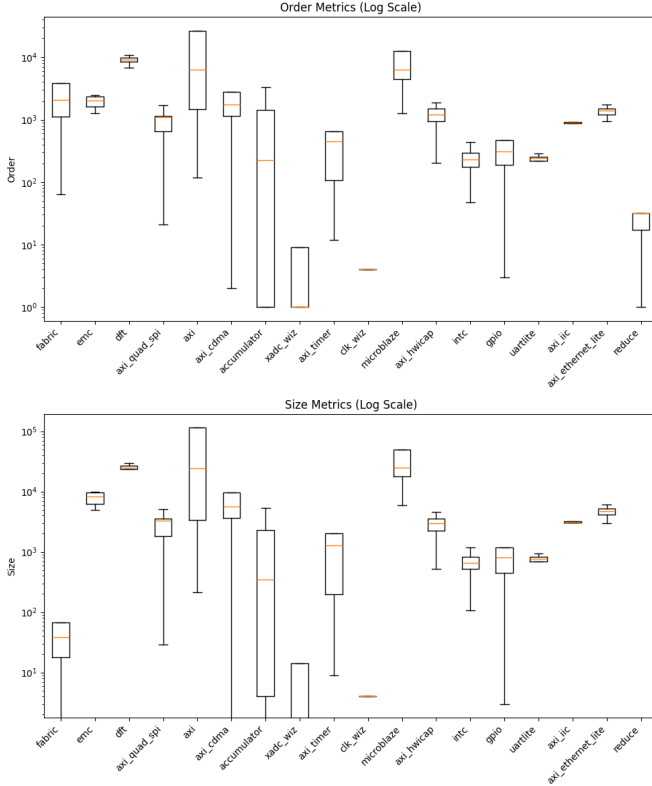


Fig. 3: Dataset 1: IP subgraph order and size in Dataset 1.

approach feasible, but the designs are not identical, making a subgraph matching approach (e.g. [7]) ineffective.

In our GNN-based IP classification framework, we translate these designs into graphs, where nodes represent BELs, such as LUTs and flip-flops, and edges represent wired connections between these elements. Based on these graph representations, Figure 4 shows the sizes of the designs in *Dataset 1*, which demonstrates the wide variety of design sizes.

From these graphs, we extract subgraphs corresponding to each IP core. Figure 3 shows the distribution of IP subgraph sizes and orders, across the thousands of instances. Note the y-axis is on a log scale. These statistics help illustrate the difficulty in recognizing these IP cores, namely that even one IP type can range multiple orders of magnitude in size, across the different configurations.

Table II demonstrates that for many IP, the large datasets allow for configurations in the test set to have been seen in the training set. In machine learning applications, it is typically problematic to have items in your test set that have been seen in the training set, as this can lead to overfitting. However, in this case it is important to acknowledge several reasons why we believe this approach is not problematic. First, although IP configurations may be the same, the circuit as a whole is

different, with no circuits in the test set being identical to any in the training set. Second, unlike graph-level classifications, where the goal is to classify the graph as a whole, we are performing node-level classification, so similar overall designs in the training and test set do not necessarily lead to overfit node-level classification. Third, connections with surrounding circuitry are different and will lead to mutations on the IP boundary. Finally, we are interested in this exact case, where IP cores may have few configurations, and want to investigate how well the model can handle this case, along with the case of IP cores with large configuration spaces. Taken together, these factors indicate that while configurations may repeat, the task remains sufficiently nontrivial and the evaluation meaningful.

We provide these datasets publicly at https://github.com/byuccl/neurocore_fpt_2025, which includes each design Tcl source file, from which can be generated a Vivado implementation checkpoint and extracted labeled graph.

V. USE CASES FOR IP DETECTION

Our two datasets were designed to help us answer our two research questions.

In *Dataset 1*, we explore designs that are comprised almost entirely of known IP that belong to the AMD/Xilinx IP library, and are present in our training graphs.

Our experimental results, discussed later in the paper, demonstrate that GNN models are well suited to this task, and can achieve high accuracy (>95%) on designs that are comprised largely of known IP. This shows that GNNs have great potential in identifying known IP in unnamed netlists, and is a substantial improvement over the prior work in [7].

However, we recognized that in practice, we may often encounter IP or general user logic that is not present in our training data. This led us to explore our second research question.

Dataset 2 was created to explore this case, and contains many more IP in the test graphs that are not present in the training data. This dataset proved problematic for some of the GNN models, and we found only GATv2 was able to achieve reasonable accuracy (~80%) on the test graphs. However, the GATv2 model is much larger and slower than both the GraphSAGE and GCN models, so we advise discretion depending on the use case.

While the use case explored by the second research question is far more general-purpose and widely applicable, we still believe the first use case (in which very high accuracy can be obtained), may still be useful in certain circumstances.

A. Use Case 1: Known IP Recognition

In this use case, we are assuming that the vast majority of the IP in the design is known and present in the training data. This may be useful for a verification use-case, where a design house creates an FPGA design, compiles it with commercial CAD tools, and then wants to verify the security of the produced bitstream. In such a case, there may be concerns about the security of the CAD tools, or the compilation process, and the company may want to manually verify that the produced

bitstream does not contain malicious circuitry. For example, the latest best practices from the NSA [11], recommend that FPGA bitstreams be verified against the original RTL design.

In this case, the design is created in-house with known 3rd-party IP or in-house RTL modules, which all could be fed into the training data for the GNN model. This training step is expensive and usually performed on the GPU, but would only need to be repeated when new IP are introduced. For incremental changes to the design, or designs using the same set of IP, a single trained model would suffice.

After training, a single inference pass of the GNN could be used to identify parts of the design that belong to the known IP, and flag nodes that do not belong to any known IP. These nodes could then be further analyzed manually to determine if they are part of any malicious circuitry. We recognize that without 100% accuracy, such a case would not be sufficient to fully verify the design, nor could it be fully automated. However, it may still be a useful first step in the process.

B. Use Case 2: IP in the Presence of Unseen IP/Logic

In the second use case, we are assuming that the design may contain IP that is not present in the training data. This may be other third-party IP that we did not train the model on, or it may be general user logic that is unique to the design. This case represents the deployment of a classification system to a more general setting, where it is not necessarily possible to train the model on all possible hardware modules that we expect to be present in the design. This could occur if we are deploying the pre-trained model to a customer, who will likely have hardware modules in their design that we did not train the model on.

This could also occur if we (or a customer of such a tool) are interested in analyzing a design of unknown origin, where it is not possible to know what IP is present in the design. This could be used in a defense application, where one is trying to reverse engineer a foreign design to determine its purpose and functionality. This could also be used in a corporate setting, where a company is trying to determine whether their IP has been stolen or used in a design without their permission.

In such a case, the model needs to be more capable of correctly classifying nodes that belong to IP that it has never been seen before.

VI. IP DETECTION WITH GNNs

Our workflow begins by converting our dataset designs from Vivado's checkpoint format into netlists and then into a graph representation suitable for consumption by a GNN. We then use Bayesian optimization to find high-performing hyperparameters for three models.

A. Netlist to Graph Conversion

While the motivation for this work is to be able to detect IP directly in a netlist reverse engineered from a bitstream, for simplicity, we do not use a bitstream-to-netlist tool, but rather export a netlist directly from Vivado in such a way that it is

similar to what would be obtained from a bitstream-to-netlist tool.

The netlists created in Vivado are dumped to a generic S-expression format that contains each BEL in the design, along with its properties, as well as the connections between the BELs (nets). We run synthesis and implementation before dumping, and our custom dump script scrapes the BELs and net information after all synthesis and implementation optimizations have been performed, including such transformations as LUT routethrus, LUT combining, pin re-ordering, logic duplication, and other optimizations.

This results in a dumped graph very similar to the content that would be recovered by a bitstream reversal tool, making it a good proxy for actual bitstream-level analysis, without the need for a bitstream-to-netlist tool. For example, in [12], McKendrick demonstrates exact structural equivalence between the bitstream, and this post-implementation data in Vivado.

These dump files are then processed to create a PyTorch Geometric graph object, which is a standard format for representing graphs in PyTorch. In this PyTorch graph, BELs are represented as nodes and nets as directed edges. The node features consist of one-hot encodings of the BEL category (ie. the type of the FPGA cell, whether it be LUT, FF, etc.) and specifically whether the BEL implements sequential or combinational logic. The discrete set of BEL types is a strong advantage to applying this work to an FPGA architecture. This work could in theory be applied to ASIC designs, but FPGAs have a significant benefit of easily coloring nodes in the graph, while an ASIC application would likely have to rely exclusively on connectivity. At this time we are not leveraging any properties of the nets (e.g. port names, pin orderings, etc.), but this could be a future area of exploration.

It is important to note that we do not use any naming information from the netlist, nor do we use any information about the hierarchy of the design. The only information we use is that which would be maintained in the bitstream itself, including the type of BEL, its properties, and the connections between them.

B. Graph Neural Network Models

Once the set of graphs are created, we are able to use fairly standard “out-of-the-box” GNN models, included in PyTorch Geometric, to perform the classification. Using our datasets, we have explored several different GNN models (GraphSAGE, GCN, GAT, GATv2, and GIN); however, our best results have been obtained from three models: GraphSAGE [13], GATv2 [14], and GCN [15].

Graph Convolutional Network (GCN) A popular GNN model that computes a normalized sum of the features of all neighbors for a given node, and then applies two transformations, one linear and one nonlinear. This linear transformation is what is trained by backpropagation.

GraphSAGE This model is commonly used for node-level classification, is focused on scalability, and designed for inductive learning, meaning it can generalize to unseen nodes. GraphSAGE assigns a feature vector to each

node for each layer in the network. The value of each vector is informed by aggregating the values from the previous layer, namely the previous vector of the node itself as well as the previous vectors of its immediate neighbors. This aggregation function is what is trained by backpropagation.

Graph Attention Network, v2 (GATv2) This model works by computing a weighted average of the neighbors of a node and then incorporating that to update the node’s value. The weighting is a learned parameter, allowing it to focus on a subset of relevant neighbors. Instead of treating all neighbors equally (like in GraphSAGE), GATv2 assigns different weights to different neighbors based on learned attention scores. GATv2 uses multi-head attention to stabilize learning and capture different aspects of the neighborhood relationships by averaging or concatenating multiple attention heads.

While GATv2 can capture more complex relationships between nodes, it can be computationally more expensive than GCN and GraphSAGE, especially for large graphs, because the attention mechanism needs to compute pairwise interactions between a node and its neighbors.

We explore several different relevant hyperparameters for the models. For GraphSAGE and GCN we explored the depth, width, activation function, dropout, and PairNorm normalization. The width controls the width of the internal feature vectors, while the depth controls the number of layers in a model. The activation function introduces the nonlinearity required for generalization, and is usually one of ReLU, LReLU, GeLU, or swish. Dropout was also explored, as it can also help with generalization, but was not found to help in these cases. Normalization can allow for deeper networks while combating oversmoothing, but our results showed minimal benefit for GraphSAGE.

In the GATv2 model, we explored similar parameters, with the addition of the number of attention heads, and learning rate decay. The heads hyperparameter is specific to attention networks, where it allows for greater ability to discriminate between the relative importance of various neighbors. With a value of eight we are focusing on learning eight different weighting schemes for each neighbor. We explored various values for this parameter, but it has a significant impact on memory usage, so we were unable to explore anything higher than 10 heads.

VII. EXPERIMENTAL METHODOLOGY AND RESULTS

We swept hyperparameters over the three different models, and employed Bayesian optimization to effectively guide the sweep towards the configurations that result in the highest accuracy on the validation portion of the dataset.

The sweep is conducted as follows: a configuration is selected according to the Bayesian model, and a run is launched. As the run progresses, it processes each graph in the training dataset, computes the loss with respect to the true labels, and incorporates this loss to update the parameters of the model. It then runs inference on each graph in the validation portion of

TABLE III: Optimal Hyperparameters

Parameter	GCN	GraphSAGE	GATv2
Width	512	24	256
Depth	10	14	6
Activation	lrelu	lrelu	GeLU
Learning Rate	5.0E-4	5.0E-4	4.4E-4
Learning Rate Decay	N/A	N/A	0.99982
Dropout	0	0	0.42
PairNorm	1	1	95
Attention Heads	N/A	N/A	8

TABLE IV: Accuracy Results

	Dataset 1	Dataset 2
GraphSAGE	97.7%	<50%
GCN	98.6%	68.4%
GATv2	97.7%	80.4%

the dataset, and computes the average accuracy for validation. The pass through both the training and validation portions of the dataset constitute an epoch. After a predetermined number of epochs, the run finishes, and the final average accuracy for validation is used to update the Bayesian model, another configuration is selected, and the process repeats.

As a frame of reference, a single run of the GraphSAGE model on 5,000 graphs for 40 epochs took 17 hours, while a run of the GATv2 model on 4,000 graphs for 25 epochs took 26 hours. The primary machine involved in training these models consisted of a 64 core AMD Ryzen Threadripper PRO 7985WX with 628 GB of RAM and a single NVIDIA RTX A6000, with 48 GB of RAM.

A. Model Accuracy

Our primary measurement of performance is accuracy, and is shown in Table IV. Accuracy is computed as the number of BELs correctly labeled over total BELs analyzed. Given we are performing prediction across several different IP, a purely random prediction would obtain very low accuracy (<10%). The GraphSAGE model achieves 97.7% accuracy across all the validation designs in Dataset 1, but performs poorly on Dataset 2. It was tested with a subset of Dataset 2 and achieved less than 50% accuracy, and so was not evaluated on the entire dataset.

The GCN model also performs well on Dataset 1, but only achieves 68.4% accuracy on Dataset 2.

The GATv2 model is the only model that performs well on Dataset 2, and achieves an average 80.4% accuracy. This was performed across all but 8 of the validation designs in Dataset 2 (these designs were skipped due to memory limitations on our GPU).

In all cases, the accuracy is only calculated on the 1,000 test graphs, separate from the 4,000 training graphs.

The final best performing hyperparameters for the three models are shown in figure III, with GCN and GraphSAGE hyperparameters shown for Dataset 1 and GATv2 hyperparameters shown for Dataset 2.

TABLE V: F1 Comparison with Previous Work

IP	Isoblaze [7]	GraphSAGE	GATv2	GCN
AXI GPIO block	68.04	95.55	47.24	91.74
Clock Wizard	0.92	1.37	3.72	13.63
AXI Interconnect	11.02	99.67	80.30	99.74
MicroBlaze	80.10	98.98	81.40	99.52

B. Accuracy versus Previous Work

In Table V, we compare F1 scores for each IP matched against the closest applicable prior results, those from [7]. We take the best results from [7]. Both GraphSAGE and GATv2 perform better on average than the work reported in [7].

Both in this work and in [7], the Clock Wizard IP went almost entirely unmatched. The clock wizard consists of six BELs, and due to the weighting of the loss function, our models have no incentive to match the Clock Wizard correctly. Interestingly, its small size and unique primitive usage makes it an easy target for a human reviewer to correctly locate and label.

The GATv2 model also matched the AXI GPIO block with less success than prior work. One factor that likely impacted this is the highly configurable nature of the IP. However, previous work had a very limited sample size of a single instance in a single design, while our work computes an average F1 score over several hundred instances. Overall, neurocore has significantly higher accuracy and vastly more thorough evaluation methodology.

C. Inference Runtime

The runtime of inference with the GATv2 model (most compute intensive model) is shown in Figure 5a. The runtime varies by number of edges in the target graph (graph size), and is about 1.5 seconds of CPU time for every 1000 edges. This translates to the largest designs taking about 10 minutes.

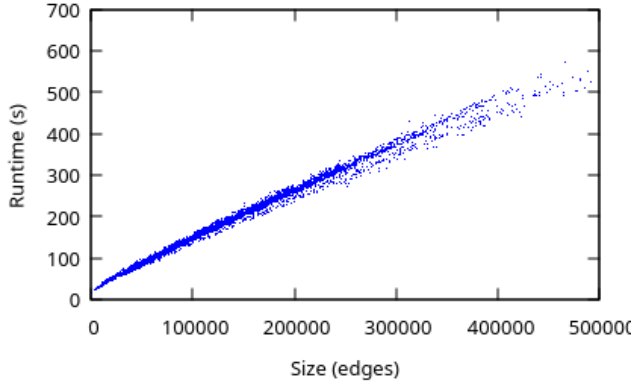
This runtime is when inference is run entirely on the CPU and the netlist is parsed in Python from plain-text. If the speed of inference does prove to be prohibitive, the computation could potentially be moved to the GPU, and the netlist could be parsed more efficiently and from a more efficient format.

Figure 5b shows the runtime of the GCN model, which is significantly faster than GATv2. All designs run in less than two minutes. Figure 5c indicates that the GraphSAGE model runs even faster than that, with even the largest designs analyzed in less than a minute. These runtimes are quite reasonable for an inference tool that would only need to be run occasionally.

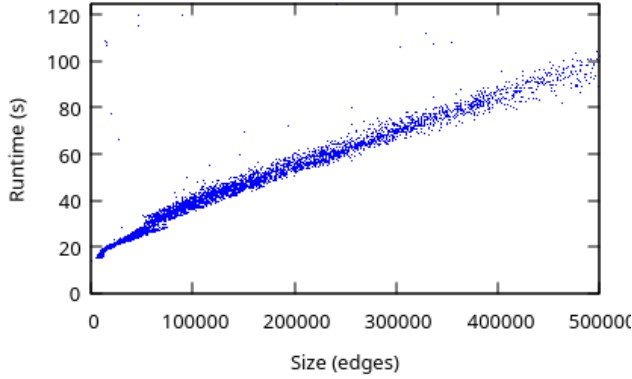
In [7], the authors report runtimes per IP, as their approach requires separate executions to search for each IP type. In contrast, the runtime of these GNN approaches scales linearly with the number of edges in the target graph, and only requires a single pass to label nodes as belonging to any of a set of IP.

D. Scalability Issues with GATv2

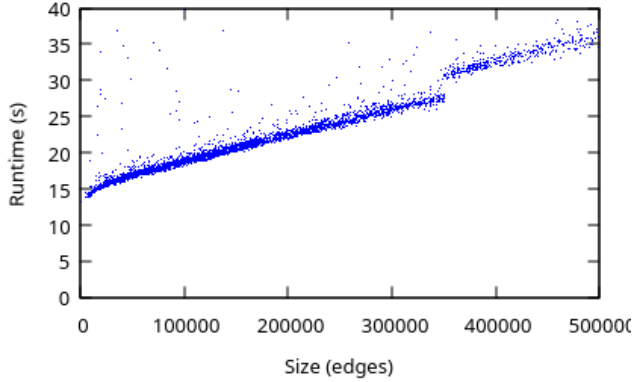
We observed that the GATv2 model scales poorly with the number of edges, and was running out of memory at around



(a) GATv2 runtime



(b) GCN runtime



(c) GraphSAGE runtime

Fig. 5: Inference runtimes

270,000 edges. This was primarily a problem on training with our setup, since training requires much more memory than inference. In [7], the largest IP core was the Microblaze with 2,238 BELs. Based on the data from Figure 3, this indicates that the size of the largest IP in prior work is between 2,000 and 20,000 edges. In addition, the target design contained only one copy of each IP core. In this work, our IP cores can be as large as 100,000 edges, and each target circuit contained between 3 and 18 randomly selected IP. This indicates that while scaling is an issue, GATv2 was still able to outperform

prior work.

Since our CPU has access to 628 GB of RAM, in contrast to the 48 GB available to the GPU, we were able to run all the way through training on the CPU, albeit at significantly reduced speeds. Since speed is not as critical during inference, this provides confidence that inference would not be unreasonable even if training is difficult.

As a stop-gap measure, we simply catch the out of memory exception and ignore those graphs. This allows us to train and validate on the largest graphs a given GPU can handle. Since training requires more memory than inference, we were only able to train on 78% of the training graphs (3,129/4,000), while we were able to compute validation accuracy across 99% of the validation graphs (992/1,000).

We explored using neighborhood sampling to keep memory usage down, but this resulted in a single training epoch taking almost 3 days. We leave scaling the GATv2 model effectively to further work. We also hope to explore in the future how well these models generalize from small graphs to larger circuits.

E. Scalability to More IP Cores

In *Dataset 1*, we trained the model, and performed inference, on 17 different IP cores. Our earlier preliminary datasets contained fewer IP cores, but achieved similar accuracy with GCN and GraphSAGE. We are not yet sure how many IP cores the model can be trained on before accuracy begins to degrade, but we plan to investigate this in the future. Should a use case require training on more IP cores than the model can effectively handle, it may be possible to train multiple models, each on a subset of the IP cores, and then combine the results. Such an approach would effectively change this to the second use case, where the model would need to distinguish between trained IP, and unknown logic.

In *Dataset 2* we train the model to detect nine different IP cores, and then test the model on designs with an additional seven IP cores. The GATv2 achieves good accuracy (80.4%), but there is still substantial room for improvement. It is possible that the accuracy could be improved by reducing the number of IP cores in the training set, with the extreme case being training on only one IP core, and having a separate model for each IP core type. We plan to investigate the trade-offs of this approach in future work.

VIII. CONCLUSION

We set out to better identify configurable IP in an unlabeled circuit. We decided on a graph neural network approach, and developed two sizeable datasets. We also characterized the performance of three models, GATv2, GraphSAGE, and GCN, against datasets representative of two different use-cases: identifying known IP in a known circuit, and identifying known IP in an arbitrary circuit.

While training of neural networks can be expensive, a trained model requires much fewer resources to actually label the circuit. In contrast to [7], all the IP are recognized in a single pass, thus mitigating even the modest inference requirements for this application.

A full solution to the problem of netlist reverse engineering would likely include multiple approaches to labeling and grouping BELs and nets in a circuit. Neurocore presents a fruitful piece of that solution, providing significant insight into the structure of a design.

REFERENCES

- [1] S. Bhunia and M. M. Tehranipoor, *Hardware Security*. Morgan Kaufmann, Oct. 30, 2018.
- [2] W. Li, Z. Wasson, and S. A. Seshia, “Reverse engineering circuits using behavioral pattern mining,” in *International Symposium on Hardware-Oriented Security and Trust*, Jun. 2012, pp. 83–88.
- [3] P. Subramanyan et al., “Reverse engineering digital circuits using structural and functional analyses,” *IEEE Transactions on Emerging Topics in Computing*, vol. 2, no. 1, pp. 63–80, Mar. 2014.
- [4] W. Li et al., “WordRev: Finding word-level structures in a sea of bit-level gates,” in *International Symposium on Hardware-Oriented Security and Trust (HOST)*, Jun. 2013, pp. 67–74.
- [5] N. Albartus, M. Hoffmann, S. Temme, L. Azriel, and C. Paar, “DANA universal dataflow analysis for gate-level netlist reverse engineering,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 309–336, Aug. 26, 2020.
- [6] R. McKendrick, C. Simpson, B. Nelson, and J. Goeders, “Leveraging FPGA primitives to improve word reconstruction during netlist reverse engineering,” in *International Conference on Field-Programmable Technology (ICFPT)*, Dec. 2022, pp. 1–5.
- [7] D. Dahl, C. Simpson, K. Faulkner, B. Nelson, and J. Goeders, “IPRec and isoblaze: Fuzzy subcircuit isomorphism for IP detection in physical netlists,” in *2023 IEEE Physical Assurance and Inspection of Electronics (PAINE)*, Huntsville, AL, USA: IEEE, Oct. 24, 2023, pp. 1–7.
- [8] A. Hastings, S. Jensen, J. Goeders, and B. Hutchings, “Using physical and functional comparisons to assure 3rd-party IP for modern FPGAs,” in *International Verification and Security Workshop (IVSW)*, Jul. 2018, pp. 80–86.
- [9] E. Cahill, B. Hutchings, and J. Goeders, “Approaches for FPGA design assurance,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 15, no. 3, 28:1–28:29, Dec. 28, 2022.
- [10] M. Hutton, J. Rose, and D. Corneil, “Automatic generation of synthetic sequential benchmark circuits,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 21, no. 8, pp. 928–940, Aug. 2002.
- [11] National Security Agency, Cybersecurity Directorate, Joint Federated Assurance Center, “DoD microelectronics: Field programmable gate array level of assurance 1 best practices,” May 13, 2024.
- [12] R. McKendrick, K. Faulkner, and J. Goeders, “Assuring netlist-to-bitstream equivalence using physical netlist generation and structural comparison,” in *International Conference on Field Programmable Technology (ICFPT)*, Dec. 2023, pp. 142–151.
- [13] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., 2017.
- [14] S. Brody, U. Alon, and E. Yahav, *How attentive are graph attention networks?* Jan. 31, 2022. arXiv: 2105.14491.
- [15] T. N. Kipf and M. Welling, *Semi-supervised classification with graph convolutional networks*, Version Number: 4, 2016.